# RobustUAVs.ai — v1

A Cross-Layer UAV Security Platform: Six Unified Corpora,
a Composed Network-to-Navigation Certificate, and a

Provenance-Enforcing Evaluation Surface

**Roger Nick Anaedevha**
Institute of Cyber Intelligent Systems, NRNU MEPhI, Moscow, Russia

Platform: `https://robustuavs.ai` · Artifact: `https://github.com/rogerpanel/RobustUAVs.ai`

August 2026

**Abstract**

RobustUAVs.ai is an open research platform for cross-layer UAV security. It exists to make one question answerable: *if an attacker stays just below a network detector's alarm threshold, will the mission still complete, and can that be proved?* Answering it requires objects no public dataset provides, so the platform supplies them.

The system spans **25 interconnected pages** in **8 sidebar groups**, served by a FastAPI control plane exposing **41 endpoints** across **16 modules**, with an Expo / React Native client of **27 screens** that runs from one source on web, iOS and Android. A registry carries **13 models** in five categories: four navigation detectors, two published baselines, one network detector, five certificates, and the composed guarantee that chains them.

Three properties distinguish it from a dashboard over a results directory. First, **nothing is hard-coded**: every number served traces to a file under `results/` or to a live call into the certificate engine, and a result that has not been produced returns HTTP 503 with the reason rather than an interpolated value. Second, **provenance is mandatory and visible**: the schema's unit of analysis is the cross-layer pairing, every pairing records how its causal coupling was established, and every page badges itself *measured* or *illustrative*. Third, the platform **reports against itself**: it states that the number of released `measured_same_platform` pairings is currently zero, and it says so on the page a reviewer would use to check.

The corpus unifies six sources totalling **431,773 events**, of which **200,610 (46.5%)** were captured on real hardware. The certificate engine implements four complementary methods and reports the fourth as a form awaiting its numeric constant rather than filling it. A live fleet simulation flies up to **12 aircraft** under **10 attack classes** with emergent mesh partitioning, and a six-phase GNSS spoofing process runs interactively. This document describes what is built, what is measured, what is modelled, and what is still blocked.

## Contents

# 1 Introduction

## 1.1 The problem the platform is built around

UAV security is studied from two directions that do not meet. Network intrusion detection inspects the traffic carried by the swarm mesh, the command-and-control link and the on-board bus, and raises an alarm when that traffic looks malicious. Certified robustness proves that a navigation model's output cannot move by more than a fixed amount when its sensor input is perturbed by no more than a given magnitude. The first stops at the alarm and says nothing about the flight that follows an intrusion it fails to catch. The second takes the perturbation as given and asks neither where it originates nor how often an adversary can produce one.

The gap between them is not academic. A detector must tolerate natural jitter to be usable, so an adversary who stays just under that tolerance is invisible by construction. What happens to the aircraft afterwards is precisely the question an operator, an insurer and a regulator need answered, and it is the question that falls between the two literatures.

## 1.2 What the platform contributes

RobustUAVs.ai is the instrument for closing that gap. It contributes four things, and the sections that follow describe each in turn.

1. A **two-layer schema** whose unit of analysis is the *cross-layer pairing*: an interval during which a network attack is active, joined to the interval of degraded navigation it induces, carrying a mandatory flag recording how strongly that causal link is evidenced (§3).
2. A **corpus** of six public sources unified under that schema by one lossless adapter each, with a detector-operating-curve harness that makes the detector's alarm threshold a swept parameter (§4).
3. A **certificate engine** implementing four complementary methods and the composition that chains a detector threshold to a mission-completion floor (§5).
4. A **web platform** that serves all of the above, adds live simulation, and enforces the provenance discipline in the user interface rather than only in the data (§2 onwards).

## 1.3 The one design rule

Every other decision in this system follows from a single rule:

> *A number is shown with its provenance, or it is not shown. When a result has not been produced, the platform says which input is missing and where that gap is tracked. It never substitutes a plausible value.*

This is why a page can return HTTP 503, why the Calibration page deliberately displays nothing, why every screen carries a *measured* or *illustrative* badge, and why the corpus page reports a count of zero against the strongest evidence class. A reviewer who cannot tell a measurement from a model cannot check the work, and a platform that blurs the two is worse than no platform.

# 2 System Architecture

## 2.1 Two-plane architecture

The system separates a **research plane** that produces numbers from a **control plane** that serves them. The separation is strict and one-directional: the control plane reads committed artefacts and never computes a research result on the request path, except for the certificate engine, which is called live because it is deterministic and fast.

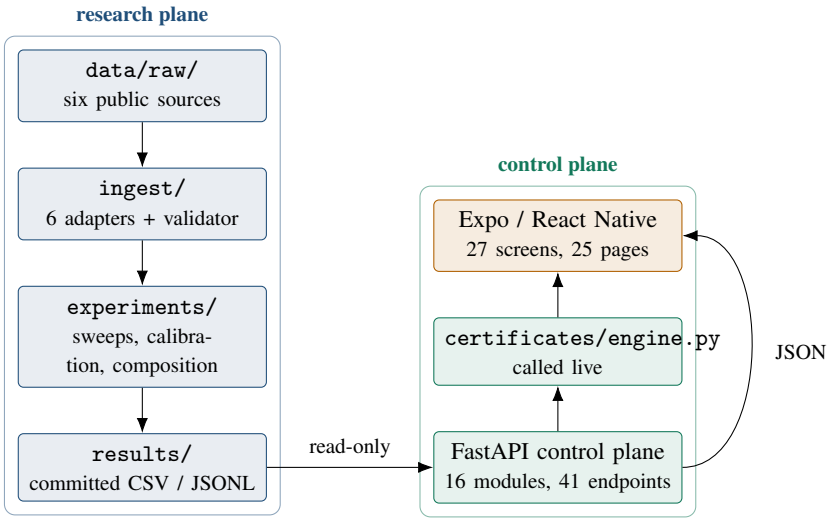


Figure 1: The two planes. Research artefacts flow one way. The control plane holds no research state of its own, which is what makes every served number traceable to a committed file.

## 2.2 Sidebar layout: 8 groups, 25 pages

| Group | Pages | Contents |
|---|---|---|
| Results | 2 | Milestones, Composition |
| UAV / Aerial Defense | 8 | UAV Monitor, Swarm Graph, Fleet Demo, GNSS Spoof Monitor, Certification, Mission Plan, Perception, Dossier |
| Evaluation | 5 | Robustness, Ablations, ROC, Statistics, Calibration |
| Analyse | 1 | Upload & Analyse |
| Build | 4 | Agent Studio, Interface stability, Mission-distribution sensitivity, Federated Learning |
| Experiments | 2 | Runs, SOC Copilot |
| Corpus | 2 | Overview, Model registry |
| Present | 1 | Cover |
| **Total** | **25** | across 8 groups |

Table 1: Navigation structure, read from `platform/mobile/src/layout/nav.js`.

Every page carries a hideable step-by-step help panel written for a reader who has not seen the page before, telling them what to look at first and what the page is evidence for. Several also carry a *tip* naming the reviewer question the page is designed to answer.

### 2.3 The 41 endpoints

| Family | Endpoints |
|---|---|
| Health and registry | `/api/health`, `/api/models`, `/api/models/{id}` |
| Results | `/api/results`, `/api/results/{name}` |
| Certificates | `/api/certify/staleness` |
| Evaluation | `/api/eval/{robustness, ablations, roc, statistics, calibration, federated, interface, mission-distribution}` |
| UAV surface | `/api/uav/{overview, certificates, dossier, fleet, fleet/catalog, fleet/reset, fleet/step, gnss/run, gnss/run/reset, gnss/run/step, gnss/sky, swarm/snapshots, mission-plan/review, perception/catalog, ew-bench/curves, ew-bench/operating-point}` |
| Runs | `/api/runners`, `/api/runs` (GET, POST), `/api/runs/{id}`, `/api/runs/{id}/cancel` |
| Copilot | `/api/copilot/{tools, tool/{name}, ask}` |
| Upload | `/api/upload/{analyse, compare, fly}` |

Table 2: The control-plane surface. Interactive documentation is served at `/docs`.

### 2.4 Two response conventions that carry meaning

**503 is not an error.** A result endpoint returns `503` when the endpoint exists but its result file has not been produced in this deployment. The client renders this as "Not produced in this deployment" and points at `PENDING_ON_DATA.md`. Reporting it as a failure would misrepresent a data gap as a bug, and those are different things to a reviewer.

**404 is a version mismatch.** A `404` means the client is asking for something this API does not have, which in practice means the bundle is newer than the backend. The client says "Client and API are out of step" and gives the redeploy command. Conflating the two sends a reader to the wrong document.

## 3 The Two-Layer Schema

The schema exists to solve one problem: six datasets describe different things, at different rates, with no common clock, and any representation that forced them into one flat table would have to discard whatever did not fit. It uses three record kinds, arranged in a hierarchy of increasing interpretation.

1. An **Event** is a single observation as its source recorded it, translated into common field names but never summarised. This layer is deliberately dumb: it asserts nothing beyond what the source file said. A `native` block preserves every unmapped source column, which is what makes ingestion provably lossless rather than merely convenient.

2. An **AttackWindow** is an interval during which one named attack was active on one layer. This layer asserts that something adversarial was happening, and when.

3. A **CrossLayerPairing** joins one network window to one autonomy window and asserts that the first *caused* the second. This is the only place in the schema where a causal claim is made, and therefore the only place carrying a mandatory evidence flag.

### 3.1 The `pairing_basis` flag

| Value | Meaning |
|---|---|
| `measured_same_platform` | Both windows recorded on one testbed. The strongest evidence. |
| `physics_model` | Coupled through a named, versioned physical relation a reviewer can inspect and challenge. |
| `synthetic_alignment` | Aligned by construction, under a stated affine clock map with a bounded error budget. |

A reviewer can filter the benchmark by evidence strength rather than trusting an unaudited join, and a claim can never silently upgrade its own provenance.

### 3.2 Three design choices

Time is always *relative* to each source's capture start, because the sources share no global clock, and cross-layer alignment happens at the window level, so a mismatch between a 50 Hz telemetry stream and a per-flow network record never forces a lossy resample. Attack labels are *unified* into a common taxonomy while the verbatim source label is retained in `label_native`. And every unmapped column is preserved.

Alignment therefore needs a stated rule and a stated error budget, since this is the step at which a benchmark could silently manufacture, or destroy, the effect it reports. Attack onsets are anchored under an affine map between the two relative clocks, with residual misalignment bounded by $e_{\text{align}} \leq 0.53$ s for every released pairing. Crucially, the quantity the certificate consumes is computed entirely within the network source, so misalignment can attach a pairing to the wrong autonomy window but cannot inflate or deflate the certified quantity itself.

## 4 The Corpus and Its Adapters

| Source | Acquisition | Layer | Events | Role |
|---|---|---|---|---|
| HCRL UAVCAN | real testbed | network, bus | 200,000 | Bus flooding, fuzzing, replay |
| UAVIDS-2025 | simulation (NS-3) | network, mesh | 122,171 | Large-scale mesh flows |
| UAV-EW-Bench-2026 | simulation (physics) | autonomy | 93,600 | The navigation anchor |
| DATAMUt replay | simulation (event) | network, mesh | 15,392 | The sweepable detector |
| UAV Attack Dataset | **real testbed** | **both** | 610 | The only source observing both layers on one airframe |
| UAV-CAS | simulation (twin) | network, mesh | 0 | Adapter ready; release file not reachable |
| **Total** | | | **431,773** | **200,610 real (46.5%)** |

Table 3: Corpus composition, regenerated by `experiments/provenance_ledger.py` so the document cannot drift from the artifact.

### 4.1 The number the platform reports against itself

Acquisition provenance is the weaker question. The stronger one is how many *pairings* rest on measurement, and there the honest answer is worse:

> *The number of released `measured_same_platform` pairings is currently **zero**. The UAV Attack Dataset release does not carry machine-readable attack on/off timestamps, so the adapter cannot emit `AttackWindow` boundaries for it and refuses to infer them. Its three flights still supply the delay-to-position rate the certificate consumes, self-referenced to each flight's own pre-attack median, so the headline result does not depend on closing this gap. But a reader who filters the artifact to the strongest evidence class will correctly retrieve nothing, and the platform says so on the Corpus page.*

### 4.2 Adapters and the validator

One adapter per source lives in `ingest/`, and `ingest/validate.py` is the gate: nothing enters `results/` unvalidated. `experiments/run_real_corpus.sh` runs the whole pipeline and is idempotent; a stage whose inputs are absent is skipped with a message rather than substituted.

*A caution that cost us a result. The third-party detector demo* appends *to its summary CSV. An early sweep harness therefore averaged every previous operating point into the current one, and the symptom was recall* increasing *with the threshold, which a threshold detector cannot exhibit.* `experiments/sweep_full.py` *now uses a fresh working directory per point. Anyone reimplementing the sweep should do the same.*

# 5 Models and Certificates

## 5.1 The registry: 13 models in five categories

| Category | Count | Members |
|---|---|---|
| navigation | 4 | `m1_ct_tgnn`, `m4_mambashield`, `m6_uc_hgp`, `m7_fedgtd` |
| baseline | 2 | `caf_cnn`, `seq2seq_tr` |
| network | 1 | `datamut` |
| certificate | 5 | `cert_gronwall`, `cert_smoothing`, `cert_staleness`, `cert_mwu`, `cert_pacbayes` |
| composition | 1 | `composed` |

Each card carries a `provenance` and a `certificate_status` field, a `constants` block, and a `caveat`. The caveat is not decoration: it is where a model states what it cannot do.

## 5.2 The composition

The chain the platform serves is

$$\theta \longmapsto \Delta(\theta) \longmapsto \delta(\theta) \longmapsto \mathrm{MCR} \geq f\big(\delta(\theta)\big),$$

read left to right: a detector's alarm threshold $\theta$ bounds the residual undetected attack $\Delta(\theta)$; the residual maps to a bounded perturbation $\delta(\theta)$ on the navigation input; and a Lipschitz–Grönwall trajectory-tube argument turns that bounded input into a guaranteed Mission-Completion-Rate floor.

*The chain is not a theorem end to end. Its first and last links are proved; the middle link,* $\Delta \mapsto \delta$*, is an* empirical calibration*. The platform and the accompanying manuscript both describe the result as a conditional guarantee under an empirically calibrated interface, and not as end-to-end certified security.*

## 5.3 Four certificates, and why one is not enough

| Certificate | Bounds | Regime it covers |
|---|---|---|
| Lipschitz–Grönwall | the trajectory | Deterministic, bounded disturbance. The only one giving a per-instant geometric envelope, so the only one that discharges the corridor test. |
| Randomized smoothing | a decision at a point | Where the budget is known only statistically. |
| PAC-Bayes | the model | The gap between the training mission mix and this sortie. |
| MWU regret | the time axis | Cumulative loss against an *adapting* jammer. Vacuous at any single instant. |

Because a trajectory, a decision, a model and a policy sequence are distinct objects, the four compose by conjunction rather than selection: a deployment reads its floor as the pointwise minimum, and the binding term identifies which assumption is closest to failing.

### 5.4 Instantiated constants

| Constant | Value | Provenance |
|---|---|---|
| Lipschitz, measured local | $L = 1.181$ | measured on visited trajectories |
| Lipschitz, global power iteration | $\hat{L} = 0.983$ | *not used*: smaller, so it would flatter every bound |
| Grönwall radius at $T_c = 1$ s | 0.1535 | derived |
| Randomized-smoothing radius | 0.44 at $\sigma = 0.25$ | measured on the trained checkpoint |
| MWU policy set | $\lvert S\rvert = 4$ | stated |
| PAC-Bayes KL | **pending** | flagged, not substituted |
| Interface rate $\gamma$ (supremum) | 1.625 m/s | 674 post-onset samples, real flights |
| Residual budget saturation | $\leq 4.84$ s | 15,392 observed hops |

The Lipschitz row is worth dwelling on. The local constant is the *larger* of the two, so it widens the trajectory tube and narrows the certified window. Quoting the global one would flatter every bound reported, which is why the platform uses the local value and says why.

## 6 The Interface Characterisation Campaign

The composition consumes $\gamma$, the metres of position error accrued per second of degraded navigation input, and the platform's most consequential empirical question is whether $\gamma$ is a single number. It is not.

### 6.1 The estimator decides the answer

Two estimators are available and they disagree qualitatively. Writing $e(t)$ for position error and $t_0$ for attack onset,

$$\gamma_{\text{cum}}(t) = \frac{e(t)}{t - t_0}, \qquad \gamma_{\Delta}(t) = \frac{e(t) - e(t_0)}{t - t_0}.$$

Onset is declared when the error crosses a detection threshold, so $e(t_0)$ is not zero: it is 0.64 m for jamming at the receiver and 9.27 m for spoofing. Dividing that pre-existing offset by a small $t - t_0$ makes $\gamma_{\text{cum}}$ diverge, reaching 8.6 m/s below one second.

> *That number is an artifact of the detector, not a property of the attack, and reporting it would have been a serious error:* 8.6 *m/s exceeds* $\gamma_{\text{req}} = 6.14$ *m/s and would have declared the framework unusable on the strength of a measurement artifact. The delay budget is responsible only for error the attacker* causes*, so* $\gamma_{\Delta}$ *is the estimator the certificate is entitled to. Both are kept in the per-sample output so the contrast can be audited.*

### 6.2 What $\gamma$ depends on

Over 674 post-onset samples, $\gamma_{\Delta}$ spans 6.1× from median to supremum and varies systematically:

| Factor | Effect | Mechanism |
|---|---|---|
| Attack type | 2.6×, spoofing > jamming | Jamming removes information and the estimator dead-reckons; spoofing injects information the estimator accepts, so error accrues at the adversary's chosen rate. |
| Satellite visibility | rises as satellites fall $14 \to 4$ | Degraded geometry, weaker correction. |
| Staleness | peaks at $\tau \in [5, 10)$ s | Error saturates while the denominator keeps growing. |
| Error signal | only 1.14× | The filter attenuates injected error far less than one might hope. |

A descriptive least-squares fit on standardised covariates explains $R^2 = 0.79$, with physically sensible signs for satellite count ($-0.22$), staleness ($-0.26$) and airspeed ($+0.12$). No $p$-values are reported: samples within a flight are strongly autocorrelated and there are two attack flights, so any nominal significance would be an artifact of the dependence structure.

### 6.3 Coverage, stated as a limit

| Factor | Status | Detail |
|---|---|---|
| Attack type | varied | jamming and spoofing |
| GNSS condition | varied, wide | satellites $14 \to 4$, HDOP $0.71 \to 4.16$, fix $3 \to 0$ |
| Staleness | varied | 0.4 to $\sim 70$ s |
| Airspeed | poor | hover; p90 of $\|\|v_{xy}\|\| < 0.65$ m/s |
| Platform | **not varied** | one PX4 / Pixhawk 4 / Holybro S500 |
| Flight mode | **not varied** | hover / loiter only |

A factor the release could not vary is reported as unvaried, never as showing no effect. The $6.1\times$ spread is therefore a *lower bound* on the variability a deployment would meet, and the two red rows need flights rather than analysis.

### 6.4 Mission-distribution sensitivity

MCR is a probability over a distribution of missions, so a bound on it describes a population and not an individual flight. The platform measures what that costs. Across nine mission distributions (three profiles $\times$ three receiver models) at a fixed defence:

- Marginalised over the whole 0–40 dB jamming sweep, the spread is **0.011**, which looks negligible.
- Conditional on attack strength, the mission profile alone moves MCR by **0.214**, and the receiver model by **0.154**.
- The jamming power at which a distribution first falls below the 0.90 reference ranges from 18.1 to 27.1 dB, a spread of **9 dB**, or roughly a factor of eight in adversary transmit power.

Averaging over the adversary hides a dependence that an operator, who flies one profile against one adversary, lives at.

## 7 The UAV / Aerial Defense Surface

### 7.1 Live fleet simulation

Up to **12 aircraft** fly a mission of nominal duration $T = 70$ s against a deadline of $(1+\kappa)T$ with $\kappa = 0.25$. The dashboard shows exactly the number selected, and the fleet is instantiated by sampling from the five reachable corpus sources rather than from a random generator.

| Class | Family | Effect |
|---|---|---|
| `none` | — | Nominal flight |
| `fgsm` | gradient | Single-step gradient sign; cheapest and weakest |
| `pgd` | gradient | Iterated with projection; the strong baseline |
| `cw` | optimisation | Carlini–Wagner |
| `deepfool` | geometric | Minimal-perturbation boundary search |
| `gaussian` | noise | Non-adversarial control |
| `gnss_spoof` | RF | Injects a false but consistent fix |
| `jam_link` | RF | Degrades mesh link quality |
| `delay_relay` | network | The class the theorem covers |
| `label_flip` | poisoning | Training-time |

Table 4: Ten attack classes, from `platform/backend/app/uav.py`.

Mesh degradation is emergent rather than scripted: link residual is the sum of a measured baseline and accumulated staleness, and a link becomes `delayed` above 0.25 s of residual, and `jammed` above 5.0 s or when the worse endpoint drops below 25 % link quality. Jamming two of eight aircraft for 30 s partitions a nine-link mesh into four clusters with two isolated, which nothing in the code instructs it to do.

***Modelled versus measured.*** *The link residuals are sampled from real corpus data; the damage rates, return-to-launch thresholds and battery model are* modelled. *`docs/simulation_architecture.md` lists the four steps that would move the modelled quantities to measured, the shortest being per-step detector inference over the already-real link residuals. Every page badges itself accordingly.*

### 7.2 GNSS spoof monitor

A six-phase interactive process rather than a static diagram: the user steps a spoofing attack through its phases, watches the satellite geometry and the estimator response, and can attach a *remark* at any step. The composed certificate is evaluated live at each phase.

### 7.3 The rest of the group

**Swarm Graph** renders mesh snapshots with a composite view at $t_4$ showing all selected aircraft together. **Certification** reads the locked certificate constants. **Mission Plan** reviews a proposed corridor and schedule against the certified window. **Perception** and **Dossier** present the model catalogue and the per-aircraft record.

## 8 Evaluation, Analysis and Copilot

### 8.1 Evaluation group

**Robustness** plots the certified floor against the detector threshold, one series per interface mapping, now including the campaign supremum at $\gamma = 1.625$ m/s. **Ablations** reports how tight $\gamma$ must be for a given corridor to certify. **ROC** gives recall and false-positive rate against the threshold, and the generalisation to any scored detector. **Statistics** carries Wilcoxon signed-rank tests with Holm correction.

**Calibration deliberately shows nothing.** Expected Calibration Error needs per-hop detector *scores*, and the detector emits a boolean, so no reliability diagram is derivable from anything committed. The page returns `available: false` with the method, the exact missing input and what would unblock it. A reviewer asking "is your detector calibrated?" gets a straight answer about why the question cannot yet be answered, which is worth more than a curve fitted to a boolean.

### 8.2 Upload & Analyse

A reviewer can upload their own data, up to **1 GB** per file across **four** slots, and run the platform's models over it. Parsing is streaming with a reservoir sample of 4,000 rows for charts, so memory is bounded regardless of file size. Multi-dataset comparison supports leave-one-out cross-validation over the delay p95, an attack-transfer matrix, and a distribution-shift measure.

### 8.3 SOC Copilot

Four LLM providers in fallback order (`anthropic`, `openai`, `gemini`, `deepseek`) with a deterministic synthetic responder last, so the page never dead-ends. A caller-supplied key is used for one request and never stored on the server or in browser storage. Clickable thumbnails fetch results from pages the user has visited, so the copilot answers from what the reviewer has actually seen.

A smoke test enforces the load-bearing invariant: a tool advertised to the model but absent from the dispatcher is a lie told to the model, and CI fails on it.

## 9 Security, Sessions and Access

### 9.1 Per-user isolation without accounts

There is deliberately **no login or registration**, because the work is under double-blind review and reviewers must not be asked to identify themselves. Isolation is achieved with an `X-Session-Id` header generated per browser, which scopes the fleet, the run list and the rate allowance.

> *The session identifier is* not *a credential and the code says so where it is defined. It prevents two reviewers in different places from disturbing each other's simulation state. It is not an authentication mechanism and must not be treated as one.*

Rate limits are applied per session: 30 LLM calls and 20 uploads per hour.

### 9.2 Ethics gate

A four-section *Ethical Use Guidance and Caution* page gates the entire application, requiring two explicit acknowledgements before access. Acceptance is recorded locally against a version string, so revising the guidance re-prompts every user.

### 9.3 Deployment hardening

Caddy terminates TLS and serves the static bundle; the API runs under systemd as `robustuavs-api`. Optional bearer tokens are supported for a deployment that needs them, supplied out of band.

## 10 Deployment

### 10.1 Stack

| Component | Detail |
|---|---|
| Host | Hetzner cloud instance, Ubuntu |
| Reverse proxy | Caddy, automatic TLS |
| API | `uvicorn` under systemd (`robustuavs-api.service`) |
| Client | Expo web export, `output: "single"`, served statically |
| Bring-up | `deploy/bootstrap.sh`, `deploy/cloud-init.yaml` |
| Release | `deploy/redeploy.sh` then `deploy/deploy.sh` |

### 10.2 One command

```
ssh robustuavs 'cd␣/srv/robustuavs/repo␣&&␣git␣checkout␣main␣\
␣␣&&␣./deploy/redeploy.sh␣--watch'
```

`redeploy.sh` fetches, then runs `deploy.sh` detached under `setsid nohup`, so a dropped SSH session does not kill the build. Ctrl-C stops watching, not the deploy.

### 10.3 Three hard-won details

**The build needs memory.** `expo export` is the memory-hungry step, and `deploy.sh` refuses to run it below `MIN_BUILD_MB` (1500 MB) without at least 1 GB of swap. On a small instance it has previously taken the box down hard enough to lose `sshd`. The guard exists because that happened.

**The origin cannot hairpin.** A verification probe from the server to its own public hostname goes out through the CDN and returns `000`, which looks like an outage when the site is fine. The probe pins the origin with `--resolve` instead.

**Fetch, do not pull.** `redeploy.sh` deliberately runs `git fetch` rather than `git pull`. `deploy.sh` already does `git reset --hard` plus `git clean -fd`, which recovers from any local mess on the deploy host. A `pull` cannot: it refuses to merge over a modified file and aborts the script, so the very reset that would have fixed the problem never runs. That once turned one dirty file into a permanently blocked deploy.

## 11 Search Visibility

SEO is embedded in the build rather than bolted on. `platform/mobile/scripts/inject-seo.mjs` injects per-route titles, descriptions, Open Graph and Twitter cards, JSON-LD (`SoftwareApplication`, `Dataset`, `ScholarlyArticle`) and Highwire citation tags at export time, with `robots.txt` and `sitemap.xml` shipped alongside. Yandex is covered as well as Google, since the work has a Russian institutional home.

*One decision is deliberately deferred. Author names in the JSON-LD and citation metadata are exactly what a double-blind venue must not be able to find from the artifact. Until the review status is settled, the choice is between stripping the author fields and adding `noindex`, and `docs/seo.md` records the trade-off rather than resolving it silently.*

## 12 Reproducibility

### 12.1 Ten minutes, no data download

```
git clone https://github.com/rogerpanel/RobustUAVs.ai && cd RobustUAVs.ai
python3 -m pip install jsonschema numpy scipy

python3 certificates/engine.py          # self-check, four certificate
    constants
python3 experiments/make_paper_figures.py
python3 experiments/provenance_ledger.py
git diff --stat results/                # expect: no changes
python3 experiments/compose_certified.py
```

The `git diff` is the load-bearing step. The manuscript `\inputs` the generated blocks directly, so a non-empty diff means the paper and the data have drifted apart, and a reviewer can falsify the "machine-generated, not transcribed" claim in one command.

### 12.2 Full reproduction

`experiments/run_real_corpus.sh` runs fetch, stage, ingest, validate, calibrate and figures. It is idempotent and skips a stage whose inputs are absent, saying so. A missing dataset degrades the run; it never silently fabricates it.

### 12.3 Anonymised artifact

`tools/anonymise.py` produces a de-identified copy for double-blind review and, more importantly, re-scans its own output against a token list defined independently of the rewrite rules, exiting non-zero on any survivor. On its first run it caught eleven real leaks the rules had missed. It deliberately does not strip `third_party/`, because that code is cited prior work rather than self-identification, and it does not touch binaries, listing them instead: PDFs carry author metadata that no text scan can see.

## 13 What Is Not Built, and What Is Blocked

### 13.1 Not built

WebSocket progress streaming (the client polls); ingestion as a background runner; the four steps in `docs/simulation_architecture.md` that would move the modelled fleet quantities to measured.

### 13.2 Blocked on external access

Tracked in `PENDING_ON_DATA.md`, which is the complete list.

| Item | What unblocks it |
|---|---|
| UAV-CAS stat CSV | The release file. Adapter is fixture-verified and the pipeline picks it up automatically. |
| Whelan attack intervals | Per-flight attack on/off times from the dataset documentation. The adapter refuses to fabricate them. |
| TEXBAT re-calibration | Real in-phase and quadrature recordings. |
| **Platform diversity** | Flights on a second and third airframe. |
| **Flight regime** | Cruise and transition segments. |
| PAC-Bayes numeric KL | *Not data-gated.* A computation over a committed checkpoint. |
| Anonymised artifact upload | Nothing external; run the scrubber and upload. |

The two bold rows are the platform's binding limitation. Everything else is downstream of the fact that no public capture observes both layers on one airframe with published attack intervals.

## 14 Roadmap

### 14.1 Three follow-on directions

Set out in full in `docs/research_roadmap_2027.md`.

1. **Certified PNT substitution.** $\gamma$ is a property of the navigation source, not of the attack, so alternative positioning modalities move the certified window without touching the proof. A cryptographically authenticated source can be jammed but not forged, which turns the certificate's admitted re-anchoring limitation into a satisfiable hypothesis.
2. **The security tax.** The residual budget is agnostic about *why* an input is stale, so post-quantum signing latency flows through the same chain an attacker's delay does. Security hardening and the adversary draw on one budget and the certificate arbitrates between them.

3. **The certificate as a compliance artifact.** The operating region already is an operational design domain and the tube radius is comparable to the SORA contingency volume. A mapping, a gap analysis, and an evidence generator.

### 14.2 Platform work each implies

A PNT source selector page; a crypto-budget calculator; a compliance-pack export. Each reuses the existing export menu, provenance badge and certificate endpoint.

## 15 Conclusion

RobustUAVs.ai v1 is a working instrument for a question that previously had no instrument: whether a mission completes when an attacker evades the detector by construction. It unifies six corpora into one representation whose unit is the cross-layer pairing, implements four certificates and the composition that chains them, and serves the result through 25 pages that badge every number with its provenance.

The platform's distinguishing property is not any single capability but its posture. It reports the number of strongest-evidence pairings as zero. It shows a Calibration page that computes nothing and explains why. It uses the larger of two Lipschitz constants because the smaller one would flatter the result. It caught, and documents, a measurement artifact that would have produced a spectacular but wrong finding. A research platform earns trust by being checkable, and being checkable means making the gaps as visible as the results.

Version 1 is the baseline. The roadmap above, and the two flight-campaign items that remain genuinely blocked, describe what version 2 has to close.

## Software Availability

| | |
|---|---|
| Platform | `https://robustuavs.ai` |
| Artifact | `https://github.com/rogerpanel/RobustUAVs.ai` |
| Corpus | Kaggle, DOI `10.34740/kaggle/dsv/18346203` |
| Licence | MIT. Third-party data stays under its own licence; none is redistributed. |

Documents worth opening, in order of usefulness to a reviewer: `docs/certified_regime_analysis.md` (the analysis that decided the headline framing, including a refuted hypothesis kept in the record); `docs/interface_campaign_results.md`; `docs/SCHEMA.md`; `results/paper_figures/README.md` (the generated provenance ledger); and `PENDING_ON_DATA.md`.